Министерство образования Республики Беларусь

Учреждение образования
«Белорусский государственный университет
информатики и радиоэлектроники»

Факультет компьютерных систем и сетей

Кафедра программного обеспечения информационных технологий

Дисциплина: Методы машинного обучения

ОТЧЕТ
по лабораторной работе №1
на тему
«Основы программирования на языке Python»

Вариант 20

Выполнил:
студент гр. 351002
Мельников Н. Н.

Проверил:
Лебедевич А. В.

Минск 2026

1 ЦЕЛЬ РАБОТЫ

Изучить основные конструкции языка программирования Python: встроенные типы данных, строки, списки, кортежи, словари и множества, условные операторы и циклы. Научиться оформлять код в виде функций и обрабатывать исключительные ситуации с помощью блоков try, except и finally. Освоить основы объектно-ориентированного программирования на Python: создание классов, наследование, переопределение методов, использование методов экземпляра, статических методов и методов класса, а также запись данных в файл.

2 ТЕОРЕТИЧЕСКИЕ СВЕДЕНИЯ

2.1 Типы данных и ввод-вывод

Python является высокоуровневым интерпретируемым языком программирования с динамической типизацией. Тип переменной определяется в момент присваивания ей значения, поэтому объявлять тип заранее не нужно. Одна и та же переменная в разные моменты времени может ссылаться на объекты разных типов.

К основным встроенным типам относятся целые числа int, числа с плавающей точкой float, логический тип bool со значениями True и False, строки str и специальное значение None. Целые числа в Python имеют неограниченную точность, поэтому переполнения при вычислениях с большими числами не происходит. Для преобразования типов используются функции int(), float(), str() и bool(). Проверить тип объекта можно функцией type() или isinstance(), причем вторая учитывает наследование и чаще применяется на практике.

Ввод данных с клавиатуры выполняется функцией input(), которая всегда возвращает строку. Если нужно получить число, результат необходимо явно преобразовать, например int(input()). Вывод выполняется функцией print(), которая принимает любое количество аргументов и по умолчанию разделяет их пробелом. Для форматированного вывода удобно использовать f-строки, в которых выражения записываются в фигурных скобках прямо внутри строки, например f"{x:.2f}" выводит число с двумя знаками после запятой.

2.2 Строки

Строка в Python представляет собой неизменяемую последовательность символов Unicode. К отдельным символам можно обращаться по индексу, а с помощью срезов вида s[start:stop:step] получать подстроки. Например, срез s[::-1] возвращает строку в обратном порядке. Для строк определено большое количество методов: lower() и upper() меняют регистр, strip() удаляет пробельные символы по краям, split() разбивает строку на список слов, join() наоборот соединяет элементы списка в одну строку, replace() заменяет подстроку, count() считает количество вхождений. Методы isalpha(), isdigit() и isalnum() позволяют проверить, из каких символов состоит строка. Так как строки неизменяемы, все эти методы не меняют исходную строку, а возвращают новую.

2.3 Коллекции

Список (list) является изменяемой упорядоченной коллекцией, которая может хранить элементы разных типов. Список записывается в квадратных скобках. Элементы можно добавлять методами append() и insert(), удалять методами remove() и pop(), сортировать методом sort() или функцией sorted().

Встроенные функции len(), sum(), min() и max() возвращают длину, сумму, минимальный и максимальный элементы. Для создания списков часто используются генераторы списков (list comprehension), например выражение [x for x in data if x % 2 == 0] создает список из четных элементов.

Кортеж (tuple) похож на список, но является неизменяемым: после создания кортежа нельзя изменить, добавить или удалить его элементы. Кортежи записываются в круглых скобках, занимают меньше памяти, чем списки, и могут использоваться в качестве ключей словаря. Функция tuple() позволяет получить кортеж из любой последовательности.

Словарь (dict) хранит пары ключ-значение и позволяет быстро получать значение по ключу. Ключами могут быть только объекты неизменяемых типов: числа, строки, кортежи. Метод get(key, default) возвращает значение по ключу или значение по умолчанию, если такого ключа нет, что удобно при подсчете количества элементов. Методы keys(), values() и items() возвращают ключи, значения и пары соответственно. Начиная с версии 3.7 словари сохраняют порядок добавления элементов.

Множество (set) является неупорядоченной коллекцией уникальных элементов. Множества поддерживают операции объединения, пересечения и разности и часто применяются для удаления повторяющихся значений.

2.4 Управляющие конструкции

Ветвление в Python реализуется оператором if с необязательными ветками elif и else. Блоки кода выделяются отступами, а не фигурными скобками, поэтому правильная расстановка отступов обязательна. Для повторения действий используются циклы for и while. Цикл for перебирает элементы любой последовательности, а в сочетании с функцией range() позволяет выполнить действие заданное число раз. Цикл while выполняется, пока истинно условие. Оператор break досрочно завершает цикл, а continue переходит к следующей итерации. Функция enumerate() позволяет при переборе получать одновременно индекс и значение элемента.

2.5 Функции

Функция объявляется ключевым словом def, после которого указывается имя функции и список параметров в скобках. Значение из функции возвращается оператором return, если он отсутствует, функция возвращает None. Параметры могут иметь значения по умолчанию, а с помощью конструкций *args и **kwargs функция может принимать произвольное количество позиционных и именованных аргументов. Переменные, созданные внутри функции, являются локальными и не видны за ее пределами. Для коротких функций, состоящих из одного выражения, используются анонимные lambda-функции, которые часто передаются в параметр key функций sorted(), min() и max().

Разбиение программы на функции делает код более понятным, позволяет использовать один и тот же фрагмент несколько раз и упрощает тестирование, так как каждую функцию можно проверить отдельно.

2.6 Обработка исключений

Исключение представляет собой ошибку, возникающую во время выполнения программы. Если исключение не обработано, программа аварийно завершается и выводит сообщение об ошибке. В Python определено много встроенных исключений. Например, ValueError возникает при передаче значения неподходящего формата, TypeError при операции над объектом неподходящего типа, ZeroDivisionError при делении на ноль, IndexError и KeyError при обращении к несуществующему индексу или ключу, FileNotFoundError при попытке открыть отсутствующий файл.

Для обработки исключений используется конструкция try/except/else/finally. В блок try помещается код, который может вызвать ошибку. Блоков except может быть несколько, каждый из них обрабатывает свой тип исключения. Блок else выполняется, если в блоке try исключений не было. Блок finally выполняется всегда, независимо от того, возникло исключение или нет, поэтому в нем обычно освобождают ресурсы, например закрывают файлы. Общий вид конструкции:

```
try:
    result = a / b
except ZeroDivisionError:
    print("Деление на ноль")
else:
    print(result)
finally:
    print("Вычисление завершено")
```

Исключение можно вызвать вручную с помощью оператора raise. Также можно создавать собственные классы исключений, наследуя их от встроенного класса Exception.

2.7 Объектно-ориентированное программирование

Python поддерживает объектно-ориентированное программирование. Класс описывает структуру и поведение объектов, а объект (экземпляр класса) является конкретной реализацией этого описания. Класс объявляется ключевым словом class. Специальный метод __init__() вызывается при создании объекта и используется для задания начальных значений атрибутов. Первым параметром любого метода экземпляра является self, через который метод получает доступ к атрибутам и другим методам конкретного объекта.

Атрибуты экземпляра задаются через self и у каждого объекта свои, а атрибуты класса объявляются в теле класса и являются общими для всех его экземпляров. Строгих модификаторов доступа в Python нет, вместо них

используются соглашения об именах. Имя с одним подчеркиванием в начале (например, _income) обозначает защищенный атрибут, который не следует использовать вне класса и его наследников. Имя с двумя подчеркиваниями (например, __value) делает атрибут приватным, при этом интерпретатор изменяет его имя, чтобы к нему нельзя было случайно обратиться снаружи класса.

Наследование позволяет создать новый класс на основе существующего. Дочерний класс получает все атрибуты и методы родительского и может добавлять свои или переопределять унаследованные. Для вызова метода родительского класса используется функция super(). Переопределение методов лежит в основе полиморфизма: объекты разных классов имеют методы с одинаковым именем, но каждый класс реализует такой метод по-своему.

Кроме обычных методов экземпляра, в классе могут быть объявлены методы класса и статические методы. Метод класса помечается декоратором @classmethod и первым параметром получает сам класс (cls), поэтому его часто используют как альтернативный конструктор или для работы с атрибутами класса. Статический метод помечается декоратором @staticmethod, не получает ни self, ни cls и по сути является обычной функцией, которая логически относится к классу.

Специальные методы, имена которых начинаются и заканчиваются двумя подчеркиваниями, позволяют определить поведение объектов при стандартных операциях. Например, __str__() задает строковое представление объекта для функции print(), __len__() вызывается функцией len(), а __add__() определяет работу оператора сложения.

2.8 Работа с файлами

Файл открывается функцией open(), которой передаются имя файла, режим работы и кодировка. Режим "r" используется для чтения, "w" для записи с предварительной очисткой файла, "a" для дозаписи в конец файла. После окончания работы файл нужно закрыть методом close(). Удобнее использовать конструкцию with, которая закрывает файл автоматически, даже если во время работы с ним произошла ошибка.

3 ИНДИВИДУАЛЬНОЕ ЗАДАНИЕ

Вариант 20.

3.1 Часть 1

1 Составить алгоритм вычисления числа π по формуле Грегори, взяв 500 членов ряда: π/4 = 1 - 1/3 + 1/5 - 1/7 + ...

2 С клавиатуры вводится текст, определить, сколько в нем гласных, а сколько согласных. В случае равенства вывести на экран все гласные буквы.

3 Дан список [1, 34, 'qwerty', 12, 13, 16, 'Love', 'Python']. Создать отдельно список из слов и отдельно из чисел. В числовом списке найти сумму и произведение всех элементов. Вывести на экран три наибольших элемента.

4 Создайте словарь из строки ' An apple a day keeps the doctor away' следующим образом: в качестве ключей возьмите символы строки, а значениями пусть будут числа, соответствующие количеству вхождений данной буквы в строку.

5 Реализуйте программу «Магазин автозапчастей», которая будет включать в себя шесть пунктов меню. У вас есть словарь, где ключ - название продукции, значение - список, который содержит состав продукции, цену и количество (шт.), которое есть в магазине. Пункты меню:

1) просмотр описания: название - описание;
2) просмотр цены: название - цена;
3) просмотр количества: название - количество;
4) вся информация;
5) покупка;
6) до свидания.

В пункте «Покупка» необходимо совершить покупку: с клавиатуры вводится название продукции и ее количество, n - выход. Посчитать цену выбранных товаров и сколько товаров осталось в изначальном списке.

6 Найдите наименьший четный элемент кортежа. Если такого нет, то выведите первый элемент.

3.2 Часть 2

Все задания выполнить с использованием функций и обработкой исключительных ситуаций при помощи блока try/except/finally.

1 Написать функцию is_prime, принимающую 1 аргумент (число от 0 до 1000) и возвращающую True, если оно простое, и False в противном случае.

2 Напишите функцию, которая будет принимать один аргумент. Если в функцию передается список, найти сумму его чисел. Если словарь, вывести первые три наибольших значения. Если число, найти сумму цифр. Если строка, посчитать количество слов. Сделать проверку со всеми этими случаями.

3 Найти в матрице первую строку, все элементы которой положительны, и сумму этих элементов.

4 Напишите программу, демонстрирующую работу try/except/finally.

3.3 Часть 3

1 Создать класс String (строка). Разработать в классе методы для работы со строками (не менее 5).

2 Базовый класс Worker (работник):

1) определить атрибуты: name, surname, position (должность), income (доход);

2) последний атрибут должен быть защищенным и ссылаться на словарь, содержащий элементы: оклад и премия, например, {"wage": wage, "bonus": bonus};

3) создать класс Position (должность) на базе класса Worker;

4) в классе Position реализовать методы получения полного имени сотрудника (get_full_name) и дохода с учетом премии (get_total_income);

5) проверить работу примера на реальных данных: создать экземпляры класса Position, передать данные, проверить значения атрибутов, вызвать методы экземпляров.

3 Создать классы «Транспортное средство», «Самолет», «Поезд», «Автомобиль». Определить время и стоимость перевозки для указанных городов и расстояний, в каждом классе реализовать переопределение метода «способ передвижения». Вывести данные о наиболее быстрой и экономичной поездке. Предусмотреть метод записи информации в файл.

4 Придумать класс самостоятельно, реализовать в нем методы экземпляра класса, статические методы, методы класса.

4 КОД РЕШЕНИЯ ИНДИВИДУАЛЬНОГО ЗАДАНИЯ

4.1 Часть 1

Листинг 4.1 - Вычисление числа π по формуле Грегори (part1/task1.py)

```
import math

n = 500
total = 0
sign = 1

for i in range(n):
    total += sign / (2 * i + 1)
    sign = -sign

pi_value = 4 * total

print("Количество членов ряда:", n)
print("Число Пи по формуле Грегори:", pi_value)
print("Значение math.pi:", math.pi)
print("Погрешность:", abs(math.pi - pi_value))
```

Листинг 4.2 - Подсчет гласных и согласных букв (part1/task2.py)

```
vowels = "аеёиоуыэюяaeiou"
not_letters = "ъь"

text = input("Введите текст: ")

vowel_count = 0
consonant_count = 0
vowel_list = []

for ch in text:
    low = ch.lower()
    if not ch.isalpha() or low in not_letters:
        continue
    if low in vowels:
        vowel_count += 1
        vowel_list.append(ch)
    else:
        consonant_count += 1

print("Количество гласных:", vowel_count)
print("Количество согласных:", consonant_count)

if vowel_count == consonant_count:
    print("Количество гласных и согласных совпадает")
    print("Гласные буквы в тексте:", " ".join(vowel_list))
```

Листинг 4.3 - Разделение списка на слова и числа (part1/task3.py)

```
data = [1, 34, 'qwerty', 12, 13, 16, 'Love', 'Python']

words = []
numbers = []

for item in data:
    if isinstance(item, str):
        words.append(item)
    else:
```

```
        numbers.append(item)

total = sum(numbers)
product = 1
for num in numbers:
    product *= num

top3 = sorted(numbers, reverse=True)[:3]

print("Исходный список:", data)
print("Список слов:", words)
print("Список чисел:", numbers)
print("Сумма элементов:", total)
print("Произведение элементов:", product)
print("Три наибольших элемента:", top3)
```

Листинг 4.4 - Словарь из символов строки (part1/task4.py)

```
text = ' An apple a day keeps the doctor away'

counts = {}
for ch in text:
    counts[ch] = counts.get(ch, 0) + 1

print("Строка:", repr(text))
print("Словарь:")
print(counts)
print()
for key, value in counts.items():
    print(f"'{key}' - {value}")
```

Листинг 4.5 - Программа «Магазин автозапчастей» (part1/task5.py)

```
shop = {
    "тормозные колодки": [
        "керамические накладки, стальное основание, комплект на одну ось",
        85.0, 20],
    "масляный фильтр": [
        "металлический корпус, фильтрующая бумага, резиновое уплотнение",
        18.5, 40],
    "свеча зажигания": [
        "керамический изолятор, иридиевый электрод",
        25.0, 60],
    "аккумулятор": [
        "свинцово-кислотный, 12 В, 60 Ач, пусковой ток 540 А",
        260.0, 8],
    "амортизатор": [
        "стальной корпус, газомасляный, передний",
        140.0, 12],
    "щетка стеклоочистителя": [
        "резиновая лента, металлический каркас, длина 600 мм",
        32.0, 25],
}


def show_menu():
    print()
    print("===== Магазин автозапчастей =====")
    print("1. Просмотр описания")
    print("2. Просмотр цены")
    print("3. Просмотр количества")
    print("4. Вся информация")
    print("5. Покупка")
```

```
    print("6. До свидания")


def show_description():
    for name, info in shop.items():
        print(f"{name} - {info[0]}")


def show_price():
    for name, info in shop.items():
        print(f"{name} - {info[1]:.2f} руб.")


def show_count():
    for name, info in shop.items():
        print(f"{name} - {info[2]} шт.")


def show_all():
    for name, info in shop.items():
        print(f"{name}: состав - {info[0]}; цена - {info[1]:.2f} руб.; "
              f"количество - {info[2]} шт.")


def buy():
    total = 0
    cart = {}
    while True:
        name = input("Введите название товара (n - выход): ").strip().lower()
        if name == "n":
            break
        if name not in shop:
            print("Такого товара нет в магазине")
            continue
        count_str = input("Введите количество: ").strip()
        if not count_str.isdigit() or int(count_str) == 0:
            print("Количество должно быть целым положительным числом")
            continue
        count = int(count_str)
        if count > shop[name][2]:
            print(f"В наличии только {shop[name][2]} шт.")
            continue
        shop[name][2] -= count
        cost = count * shop[name][1]
        total += cost
        cart[name] = cart.get(name, 0) + count
        print(f"Добавлено в корзину: {name} x {count} = {cost:.2f} руб.")

    if cart:
        print()
        print("Ваша покупка:")
        for name, count in cart.items():
            print(f"{name} - {count} шт.")
        print(f"Итого к оплате: {total:.2f} руб.")
        print()
        print("Осталось в магазине:")
        show_count()
    else:
        print("Покупка не совершена")


while True:
    show_menu()
    choice = input("Выберите пункт меню: ").strip()
```

```
    if choice == "1":
        show_description()
    elif choice == "2":
        show_price()
    elif choice == "3":
        show_count()
    elif choice == "4":
        show_all()
    elif choice == "5":
        buy()
    elif choice == "6":
        print("До свидания!")
        break
    else:
        print("Такого пункта нет, попробуйте еще раз")
```

Листинг 4.6 - Наименьший четный элемент кортежа (part1/task6.py)

```
line = input("Введите целые числа через пробел: ")
numbers = tuple(int(x) for x in line.split())

print("Кортеж:", numbers)

evens = [x for x in numbers if x % 2 == 0]

if not numbers:
    print("Кортеж пустой")
elif evens:
    print("Наименьший четный элемент:", min(evens))
else:
    print("Четных элементов нет, первый элемент:", numbers[0])
```

4.2 Часть 2

Листинг 4.7 - Функция is_prime (part2/task1.py)

```
def is_prime(n):
    if isinstance(n, bool) or not isinstance(n, int):
        raise TypeError("аргумент должен быть целым числом")
    if n < 0 or n > 1000:
        raise ValueError("число должно быть в диапазоне от 0 до 1000")
    if n < 2:
        return False
    i = 2
    while i * i <= n:
        if n % i == 0:
            return False
        i += 1
    return True


def read_number(text):
    try:
        return int(text)
    except ValueError:
        raise ValueError("нужно ввести целое число")


def main():
    primes = [x for x in range(101) if is_prime(x)]
    print("Простые числа от 0 до 100:", primes)
    print()
    while True:
```

```
        s = input("Введите число от 0 до 1000 (q - выход): ").strip()
        if s.lower() == "q":
            print("Выход из программы")
            break
        try:
            num = read_number(s)
            result = is_prime(num)
            print(f"is_prime({num}) = {result}")
        except (ValueError, TypeError) as e:
            print("Ошибка:", e)
        finally:
            print("-" * 40)


main()
```

Листинг 4.8 - Функция обработки аргумента разного типа (part2/task2.py)

```
def is_number(x):
    return isinstance(x, (int, float)) and not isinstance(x, bool)


def process(value):
    if isinstance(value, list):
        total = 0
        for item in value:
            if is_number(item):
                total += item
        return total
    if isinstance(value, dict):
        try:
            values = sorted(value.values(), reverse=True)
        except TypeError:
            raise TypeError("значения словаря нельзя сравнить между собой")
        return values[:3]
    if is_number(value):
        return sum(int(d) for d in str(value) if d.isdigit())
    if isinstance(value, str):
        return len(value.split())
    raise TypeError(f"тип {type(value).__name__} не поддерживается")


def describe(value):
    if isinstance(value, list):
        return "Список, сумма чисел"
    if isinstance(value, dict):
        return "Словарь, три наибольших значения"
    if is_number(value):
        return "Число, сумма цифр"
    if isinstance(value, str):
        return "Строка, количество слов"
    return "Неизвестный тип"


tests = [
    [4, 15, "abc", 2.5, 10, True, "7"],
    {"январь": 120, "февраль": 95, "март": 140, "апрель": 87, "май": 160},
    {"a": 3, "b": 1},
    98765,
    -4071,
    "Python простой и удобный язык программирования",
    "",
    {"x": 10, "y": "двадцать"},
    (1, 2, 3),
```

```
    None,
]

for t in tests:
    print("Аргумент:", repr(t))
    try:
        result = process(t)
        print(describe(t) + ":", result)
    except TypeError as e:
        print("Ошибка:", e)
    finally:
        print("-" * 50)
```

Листинг 4.9 - Поиск строки матрицы с положительными элементами (part2/task3.py)

```
import random


def read_int(text):
    value = input(text)
    try:
        return int(value)
    except ValueError:
        raise ValueError(f"'{value}' не является целым числом")


def create_random_matrix(rows, cols):
    matrix = []
    for _ in range(rows):
        matrix.append([random.randint(-5, 20) for _ in range(cols)])
    return matrix


def input_matrix(rows, cols):
    matrix = []
    for i in range(rows):
        line = input(f"Строка {i + 1}: ").split()
        if len(line) != cols:
            raise ValueError(f"в строке должно быть {cols} элементов")
        try:
            matrix.append([int(x) for x in line])
        except ValueError:
            raise ValueError("элементы матрицы должны быть целыми числами")
    return matrix


def print_matrix(matrix):
    for row in matrix:
        print("".join(f"{x:5}" for x in row))


def find_positive_row(matrix):
    for i, row in enumerate(matrix):
        if all(x > 0 for x in row):
            return i, sum(row)
    return None


def main():
    try:
        rows = read_int("Введите количество строк: ")
        cols = read_int("Введите количество столбцов: ")
```

```
        if rows <= 0 or cols <= 0:
            raise ValueError("размеры матрицы должны быть больше нуля")
        print("1 - ввести матрицу вручную, 2 - заполнить случайно")
        mode = input("Ваш выбор: ").strip()
        if mode == "1":
            matrix = input_matrix(rows, cols)
        elif mode == "2":
            matrix = create_random_matrix(rows, cols)
        else:
            raise ValueError("нужно выбрать 1 или 2")
        print("Матрица:")
        print_matrix(matrix)
        result = find_positive_row(matrix)
        if result is None:
            print("В матрице нет строки, все элементы которой положительны")
        else:
            index, total = result
            print(f"Первая строка с положительными элементами: {index + 1}")
            print("Элементы строки:", matrix[index])
            print("Сумма элементов:", total)
    except ValueError as e:
        print("Ошибка:", e)
    finally:
        print("Работа программы завершена")


main()
```

Листинг 4.10 - Демонстрация работы try/except/finally (part2/task4.py)

```
class NegativeValueError(Exception):
    pass


def divide(a, b):
    return a / b


def square_root(x):
    if x < 0:
        raise NegativeValueError(
            "нельзя извлечь корень из отрицательного числа")
    return x ** 0.5


def read_file(filename):
    f = None
    try:
        f = open(filename, "r", encoding="utf-8")
        text = f.read()
        print("Содержимое файла:", text)
    except FileNotFoundError:
        print(f"Ошибка: файл '{filename}' не найден")
    finally:
        if f is not None:
            f.close()
            print("Файл закрыт")
        print("finally: работа с файлом завершена")


def get_element(items, index):
    try:
        return items[index]
    except IndexError:
```

```
        print(f"Ошибка: индекса {index} нет в списке")
        return None
    finally:
        print(f"finally: обращение к элементу с индексом {index}")


print("1. Деление двух чисел")
while True:
    a_str = input("Введите делимое (q - выход): ").strip()
    if a_str.lower() == "q":
        break
    b_str = input("Введите делитель: ").strip()
    try:
        a = float(a_str)
        b = float(b_str)
        result = divide(a, b)
    except ValueError:
        print("Ошибка: введено не число")
    except ZeroDivisionError:
        print("Ошибка: деление на ноль")
    else:
        print("Результат деления:", result)
    finally:
        print("finally: операция деления завершена")
    print()

print()
print("2. Собственное исключение")
for x in [16, -9]:
    try:
        print(f"Корень из {x} =", square_root(x))
    except NegativeValueError as e:
        print("Ошибка:", e)
    finally:
        print("finally: проверка числа", x, "завершена")

print()
print("3. Обращение к элементу списка")
data = [10, 20, 30]
print("Элемент:", get_element(data, 1))
print("Элемент:", get_element(data, 5))

print()
print("4. Чтение файла")
with open("test.txt", "w", encoding="utf-8") as f:
    f.write("Привет, Python!")
read_file("test.txt")
read_file("missing.txt")
```

4.3 Часть 3

Листинг 4.11 - Класс String (part3/task1.py)

```
class String:
    vowels = "аеёиоуыэюяaeiou"

    def __init__(self, text=""):
        self.text = str(text)

    def __str__(self):
        return self.text

    def __len__(self):
```

```
        return len(self.text)

    def __add__(self, other):
        return String(self.text + str(other))

    def reverse(self):
        return String(self.text[::-1])

    def count_words(self):
        return len(self.text.split())

    def count_vowels(self):
        return sum(1 for ch in self.text.lower() if ch in self.vowels)

    def is_palindrome(self):
        s = "".join(ch.lower() for ch in self.text if ch.isalnum())
        return len(s) > 0 and s == s[::-1]

    def capitalize_words(self):
        words = [word.capitalize() for word in self.text.split()]
        return String(" ".join(words))

    def remove_spaces(self):
        return String("".join(self.text.split()))

    def replace_word(self, old, new):
        return String(self.text.replace(old, new))

    def char_frequency(self):
        freq = {}
        for ch in self.text.lower():
            if ch.isalpha():
                freq[ch] = freq.get(ch, 0) + 1
        return freq

    def longest_word(self):
        words = self.text.split()
        if not words:
            return ""
        return max(words, key=len)


s = String(input("Введите строку: "))

print("Строка:", s)
print("Длина строки:", len(s))
print("Строка наоборот:", s.reverse())
print("Количество слов:", s.count_words())
print("Количество гласных:", s.count_vowels())
print("Является палиндромом:", s.is_palindrome())
print("Каждое слово с заглавной буквы:", s.capitalize_words())
print("Без пробелов:", s.remove_spaces())
print("Самое длинное слово:", s.longest_word())
print("Частота букв:", s.char_frequency())

old = input("Какое слово заменить: ")
new = input("На какое слово заменить: ")
print("После замены:", s.replace_word(old, new))
print("Сложение строк:", s + "!!!")

p = String("А роза упала на лапу Азора")
print()
print("Строка:", p)
print("Является палиндромом:", p.is_palindrome())
```

Листинг 4.12 - Классы Worker и Position (part3/task2.py)

```
class Worker:
    def __init__(self, name, surname, position, wage, bonus):
        self.name = name
        self.surname = surname
        self.position = position
        self._income = {"wage": wage, "bonus": bonus}


class Position(Worker):
    def get_full_name(self):
        return f"{self.name} {self.surname}"

    def get_total_income(self):
        return self._income["wage"] + self._income["bonus"]


workers = [
    Position("Иван", "Петров", "инженер-программист", 3200, 800),
    Position("Анна", "Смирнова", "аналитик данных", 2900, 650),
    Position("Сергей", "Ковалев", "инженер по тестированию", 2100, 350),
]

for w in workers:
    print("Имя:", w.name)
    print("Фамилия:", w.surname)
    print("Должность:", w.position)
    print("Доход (атрибут _income):", w._income)
    print("Полное имя:", w.get_full_name())
    print("Доход с учетом премии:", w.get_total_income(), "руб.")
    print("-" * 45)

best = max(workers, key=lambda x: x.get_total_income())
print("Самый высокий доход у сотрудника:", best.get_full_name(),
      "-", best.get_total_income(), "руб.")
```

Листинг 4.13 - Классы транспортных средств (part3/task3.py)

```
def format_time(hours):
    total_minutes = round(hours * 60)
    return f"{total_minutes // 60} ч {total_minutes % 60} мин"


class Vehicle:
    def __init__(self, name, speed, extra_time=0):
        self.name = name
        self.speed = speed
        self.extra_time = extra_time

    def move(self):
        return "способ передвижения не задан"

    def get_time(self, distance):
        return distance / self.speed + self.extra_time

    def get_cost(self, distance):
        return 0

    def get_info(self, city_from, city_to, distance):
        time = format_time(self.get_time(distance))
        cost = self.get_cost(distance)
        return (f"{self.name} ({self.move()}): {city_from} - {city_to}, "
                f"{distance} км, время {time}, стоимость {cost:.2f} руб.")
```

```
    def write_to_file(self, filename, city_from, city_to, distance):
        with open(filename, "a", encoding="utf-8") as f:
            f.write(self.get_info(city_from, city_to, distance) + "\n")


class Airplane(Vehicle):
    def __init__(self):
        super().__init__("Самолет", 750, 2.5)
        self.base_price = 60
        self.price_per_km = 0.2

    def move(self):
        return "летит по воздуху"

    def get_cost(self, distance):
        return self.base_price + distance * self.price_per_km


class Train(Vehicle):
    def __init__(self):
        super().__init__("Поезд", 90, 0.3)
        self.price_per_km = 0.07

    def move(self):
        return "едет по железной дороге"

    def get_cost(self, distance):
        return distance * self.price_per_km


class Car(Vehicle):
    def __init__(self):
        super().__init__("Автомобиль", 80)
        self.fuel_consumption = 7.5
        self.fuel_price = 2.6

    def move(self):
        return "едет по автомобильной дороге"

    def get_cost(self, distance):
        return distance / 100 * self.fuel_consumption * self.fuel_price


def compare(vehicles, city_from, city_to, distance, filename):
    print(f"Маршрут: {city_from} - {city_to} ({distance} км)")
    for v in vehicles:
        print("  " + v.get_info(city_from, city_to, distance))
        v.write_to_file(filename, city_from, city_to, distance)

    fastest = min(vehicles, key=lambda v: v.get_time(distance))
    cheapest = min(vehicles, key=lambda v: v.get_cost(distance))
    best_time = format_time(fastest.get_time(distance))
    best_cost = cheapest.get_cost(distance)
    line1 = f"Самая быстрая поездка: {fastest.name}, {best_time}"
    line2 = (f"Самая экономичная поездка: {cheapest.name}, "
             f"{best_cost:.2f} руб.")
    print("  " + line1)
    print("  " + line2)
    print()
    with open(filename, "a", encoding="utf-8") as f:
        f.write(line1 + "\n")
        f.write(line2 + "\n\n")
```

```
filename = "trips.txt"
open(filename, "w", encoding="utf-8").close()

vehicles = [Airplane(), Train(), Car()]
routes = [
    ("Минск", "Брест", 350),
    ("Минск", "Москва", 715),
    ("Минск", "Вильнюс", 185),
]

for city_from, city_to, distance in routes:
    compare(vehicles, city_from, city_to, distance, filename)

print("Добавьте свой маршрут")
city_from = input("Город отправления: ").strip()
city_to = input("Город прибытия: ").strip()
distance_str = input("Расстояние, км: ").strip()
if city_from and city_to and distance_str.isdigit() and int(distance_str):
    print()
    compare(vehicles, city_from, city_to, int(distance_str), filename)
else:
    print("Маршрут введен неверно")

print(f"Информация о поездках записана в файл {filename}")
```

Листинг 4.14 - Класс Student (part3/task4.py)

```
class Student:
    count = 0
    min_grade = 1
    max_grade = 10

    def __init__(self, name, group, grades=None):
        self.name = name
        self.group = group
        self.grades = []
        if grades:
            for g in grades:
                self.add_grade(g)
        Student.count += 1

    def add_grade(self, grade):
        if not Student.is_valid_grade(grade):
            raise ValueError(f"оценка {grade} должна быть от "
                             f"{Student.min_grade} до {Student.max_grade}")
        self.grades.append(grade)

    def average(self):
        if not self.grades:
            return 0
        return round(sum(self.grades) / len(self.grades), 2)

    def info(self):
        avg = self.average()
        return (f"{self.name}, гр. {self.group}, оценки: {self.grades}, "
                f"средний балл: {avg} ({Student.grade_to_text(avg)})")

    @staticmethod
    def is_valid_grade(grade):
        if not isinstance(grade, int):
            return False
        return Student.min_grade <= grade <= Student.max_grade
```

```
    @staticmethod
    def grade_to_text(grade):
        if grade >= 9:
            return "отлично"
        if grade >= 7:
            return "хорошо"
        if grade >= 4:
            return "удовлетворительно"
        return "неудовлетворительно"

    @classmethod
    def from_string(cls, line):
        name, group, grades = line.split(";")
        marks = [int(x) for x in grades.split()]
        return cls(name.strip(), group.strip(), marks)

    @classmethod
    def get_count(cls):
        return cls.count


s1 = Student("Козлов Андрей", "351002", [8, 9, 7, 10])
s2 = Student.from_string("Морозова Ольга; 351002; 6 7 5 8")
s3 = Student("Новиков Павел", "351003")
s3.add_grade(9)
s3.add_grade(10)

print("Методы экземпляра:")
for s in [s1, s2, s3]:
    print(s.info())

print()
print("Статические методы:")
print("Оценка 7 допустима:", Student.is_valid_grade(7))
print("Оценка 12 допустима:", Student.is_valid_grade(12))
print("Оценка 9.3 словами:", Student.grade_to_text(9.3))

print()
print("Методы класса:")
print("Создано студентов:", Student.get_count())

print()
try:
    s1.add_grade(15)
except ValueError as e:
    print("Ошибка:", e)

best = max([s1, s2, s3], key=lambda s: s.average())
print("Лучший средний балл:", best.name, "-", best.average())
```

5 РЕЗУЛЬТАТЫ ВЫПОЛНЕНИЯ

5.1 Часть 1

На рисунке 5.1 показан результат вычисления числа π по формуле Грегори. При 500 членах ряда полученное значение отличается от math.pi примерно на 0,002, так как ряд сходится медленно.

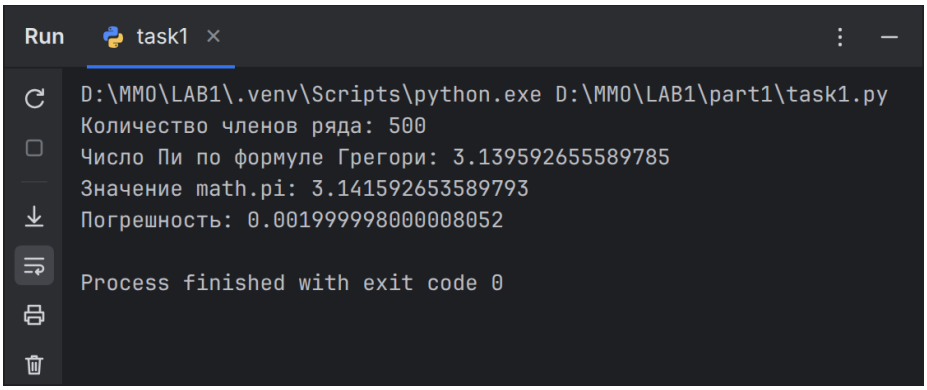


Рисунок 5.1 - Вычисление числа π по формуле Грегори

На рисунках 5.2 и 5.3 показана работа программы подсчета гласных и согласных букв. Во втором случае количество гласных и согласных совпадает, поэтому программа дополнительно выводит все гласные буквы текста.

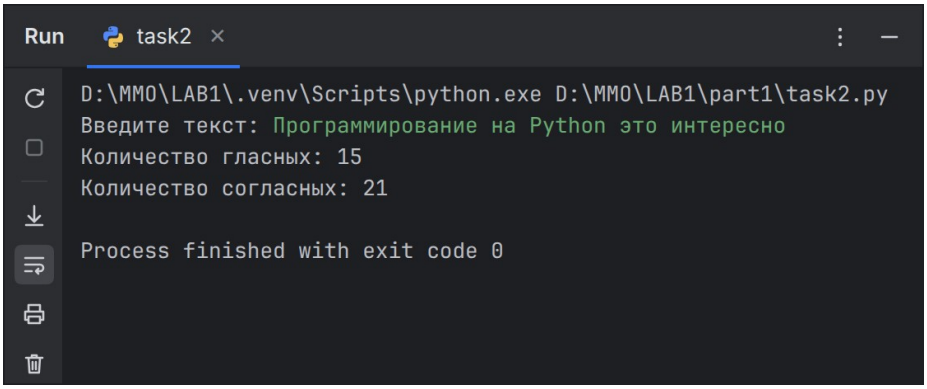


Рисунок 5.2 - Подсчет гласных и согласных, количество не совпадает

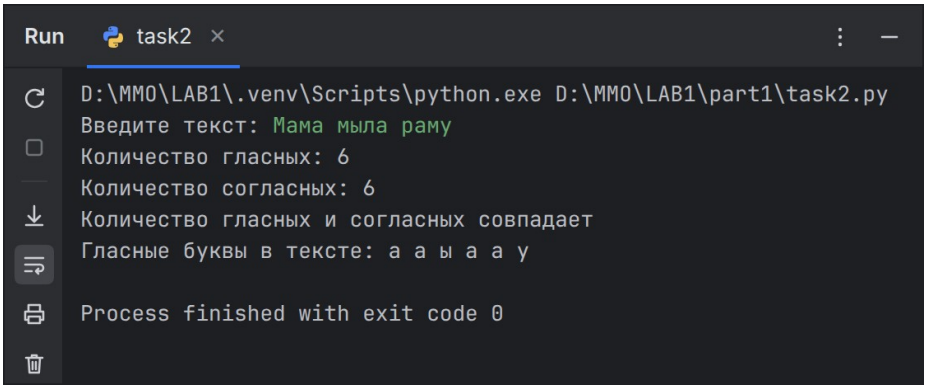


Рисунок 5.3 - Подсчет гласных и согласных, количество совпадает

Результат разделения списка на слова и числа приведен на рисунке 5.4, словарь, построенный из символов строки, на рисунке 5.5.

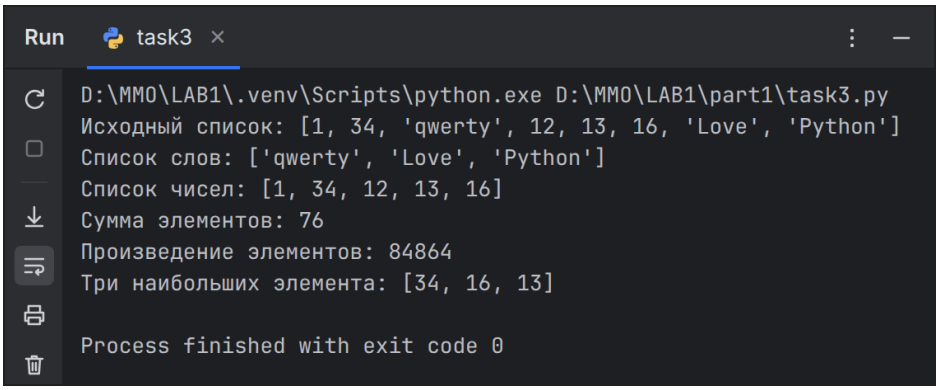


Рисунок 5.4 - Разделение списка на слова и числа

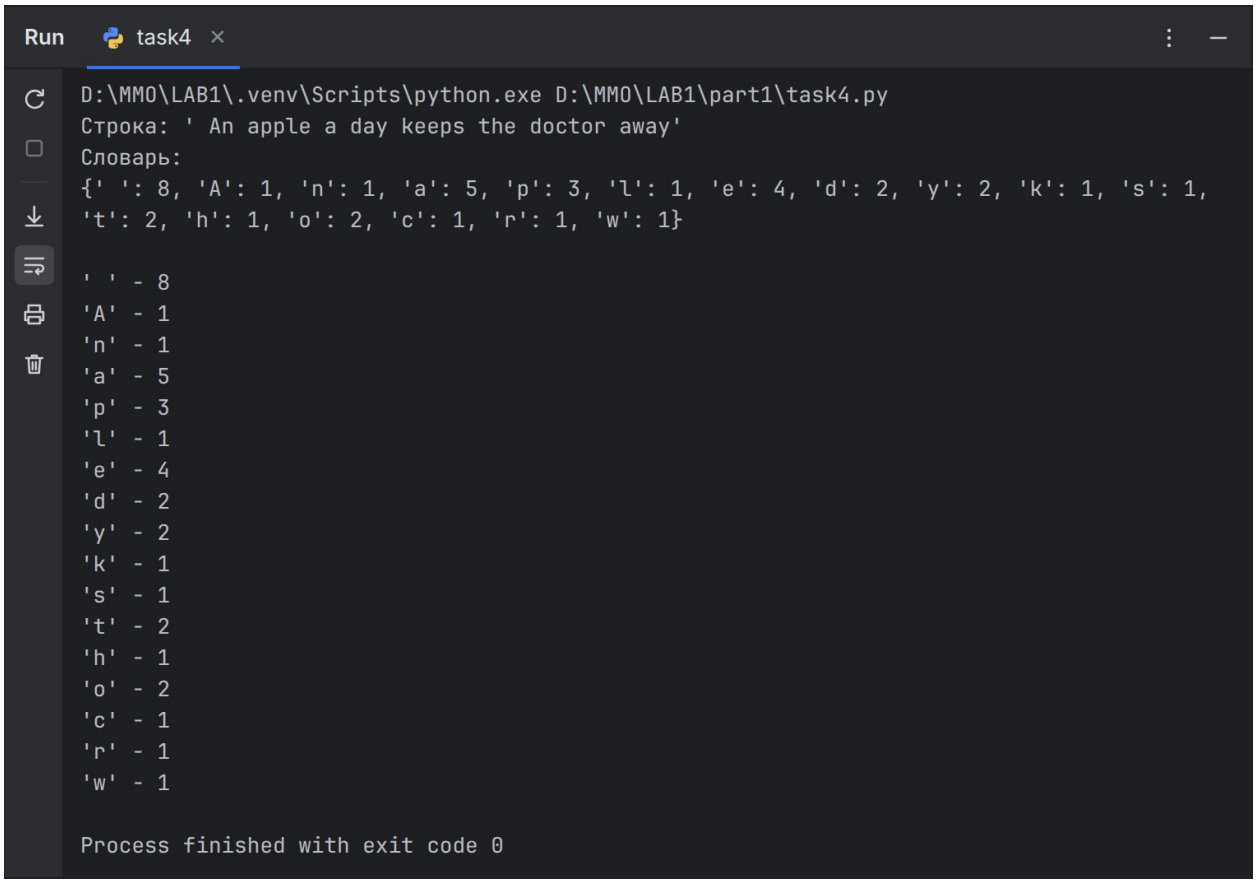


Рисунок 5.5 - Словарь количества вхождений символов

На рисунках 5.6 - 5.9 показана работа программы «Магазин автозапчастей»: просмотр описания, цены, количества и всей информации, покупка нескольких товаров с проверкой неправильного ввода, выбор несуществующего пункта меню и выход из программы.

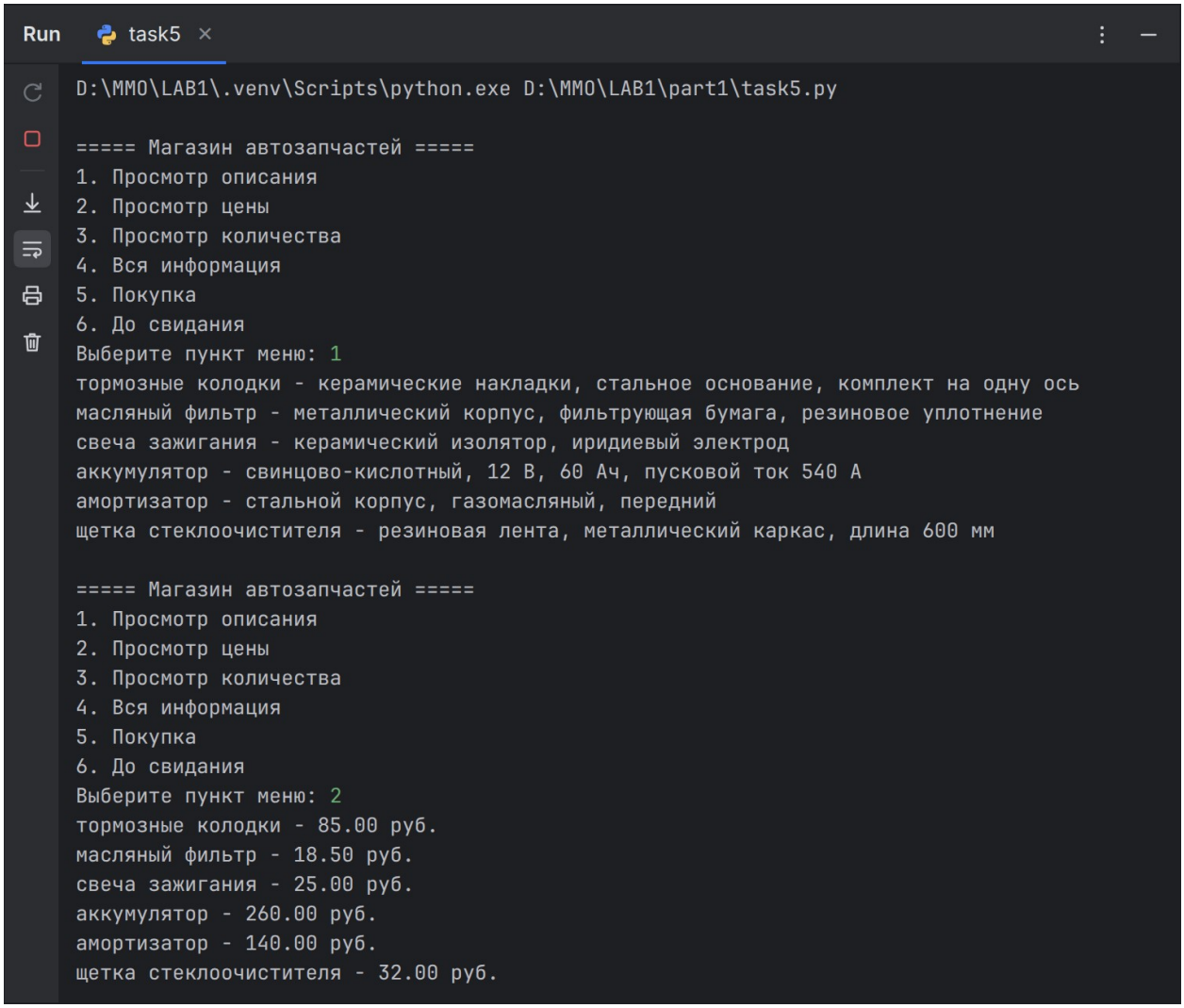


Рисунок 5.6 - Магазин автозапчастей, пункты меню 1 и 2

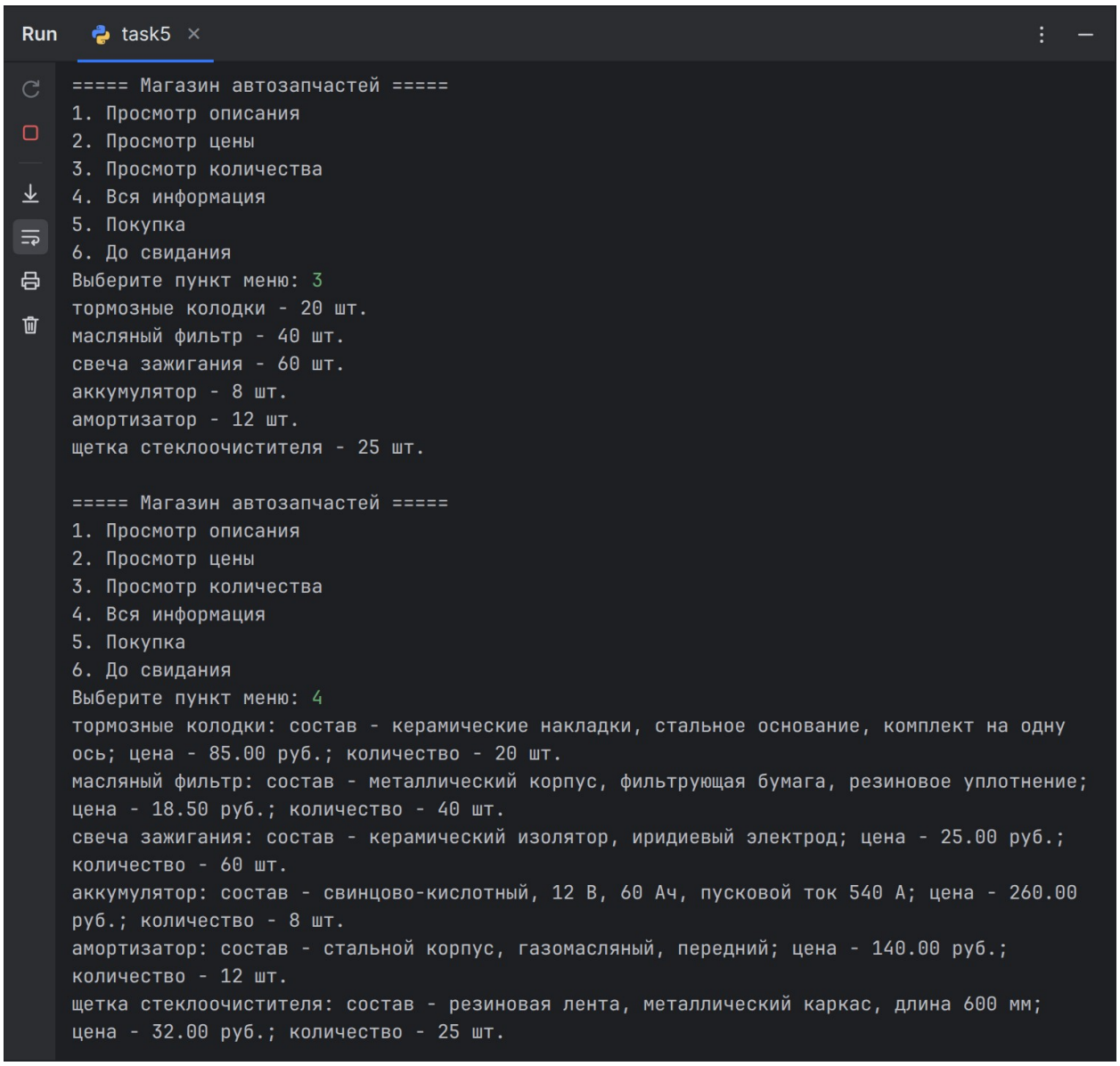


Рисунок 5.7 - Магазин автозапчастей, пункты меню 3 и 4

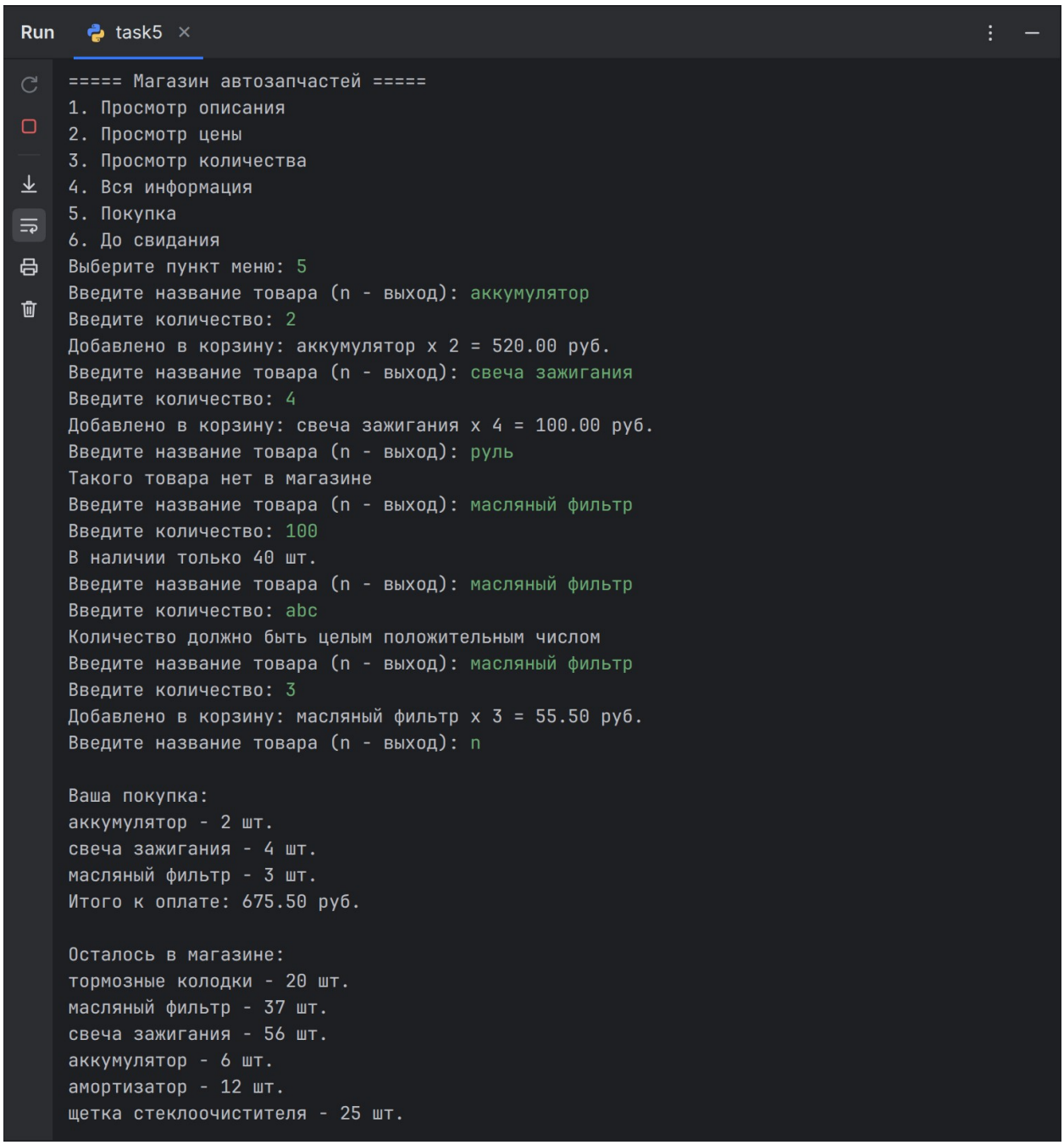


Рисунок 5.8 - Магазин автозапчастей, покупка товаров

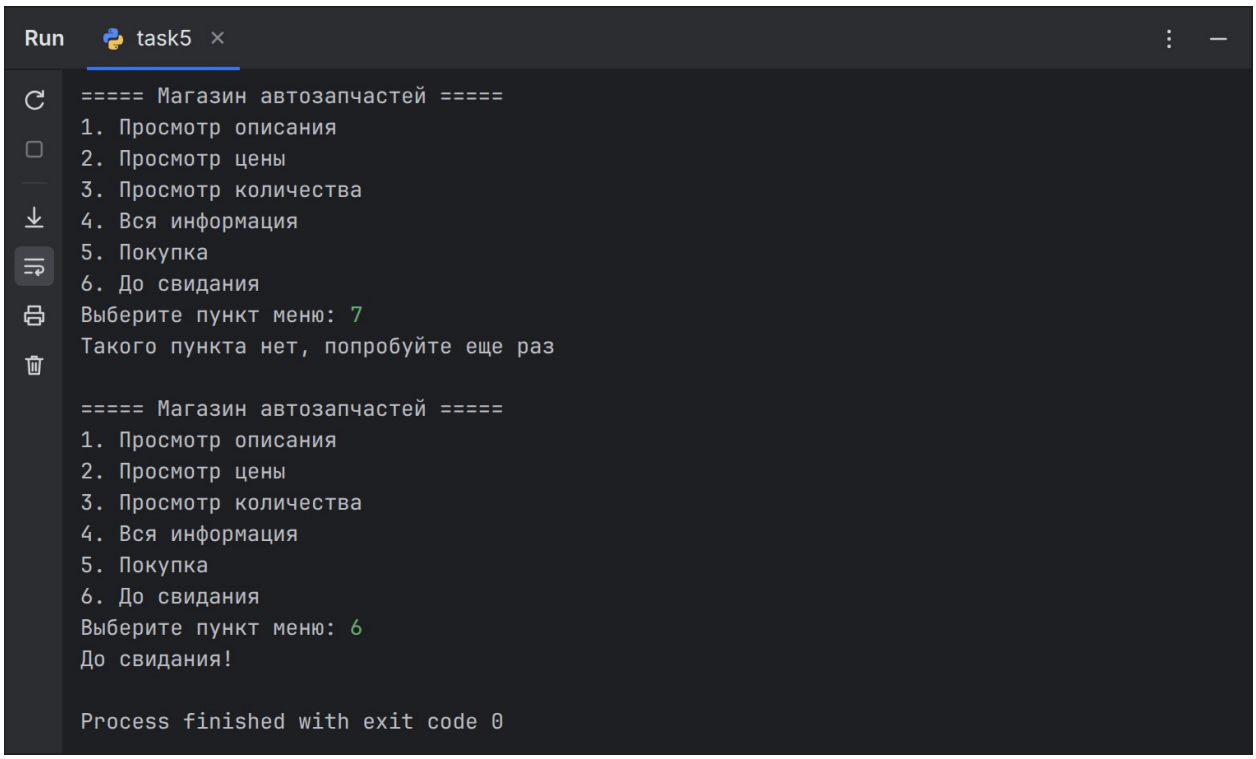


Рисунок 5.9 - Магазин автозапчастей, неверный пункт и выход

На рисунках 5.10 и 5.11 показан поиск наименьшего четного элемента кортежа для случаев, когда четные элементы есть и когда их нет.

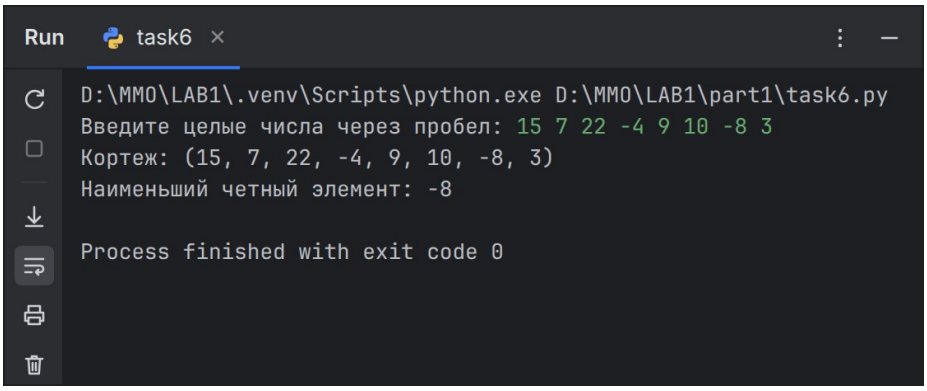


Рисунок 5.10 - Наименьший четный элемент кортежа

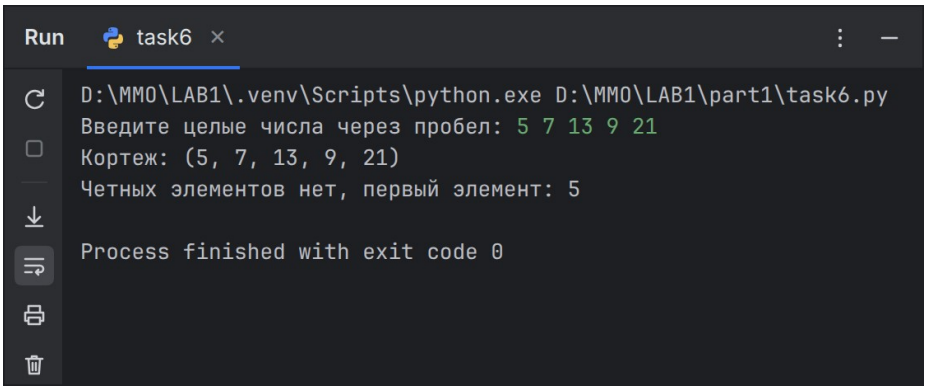


Рисунок 5.11 - Кортеж без четных элементов

5.2 Часть 2

На рисунке 5.12 показана работа функции is_prime. Кроме проверки введенных чисел, программа выводит все простые числа до 100. При вводе числа вне диапазона или не числа выводится сообщение об ошибке.

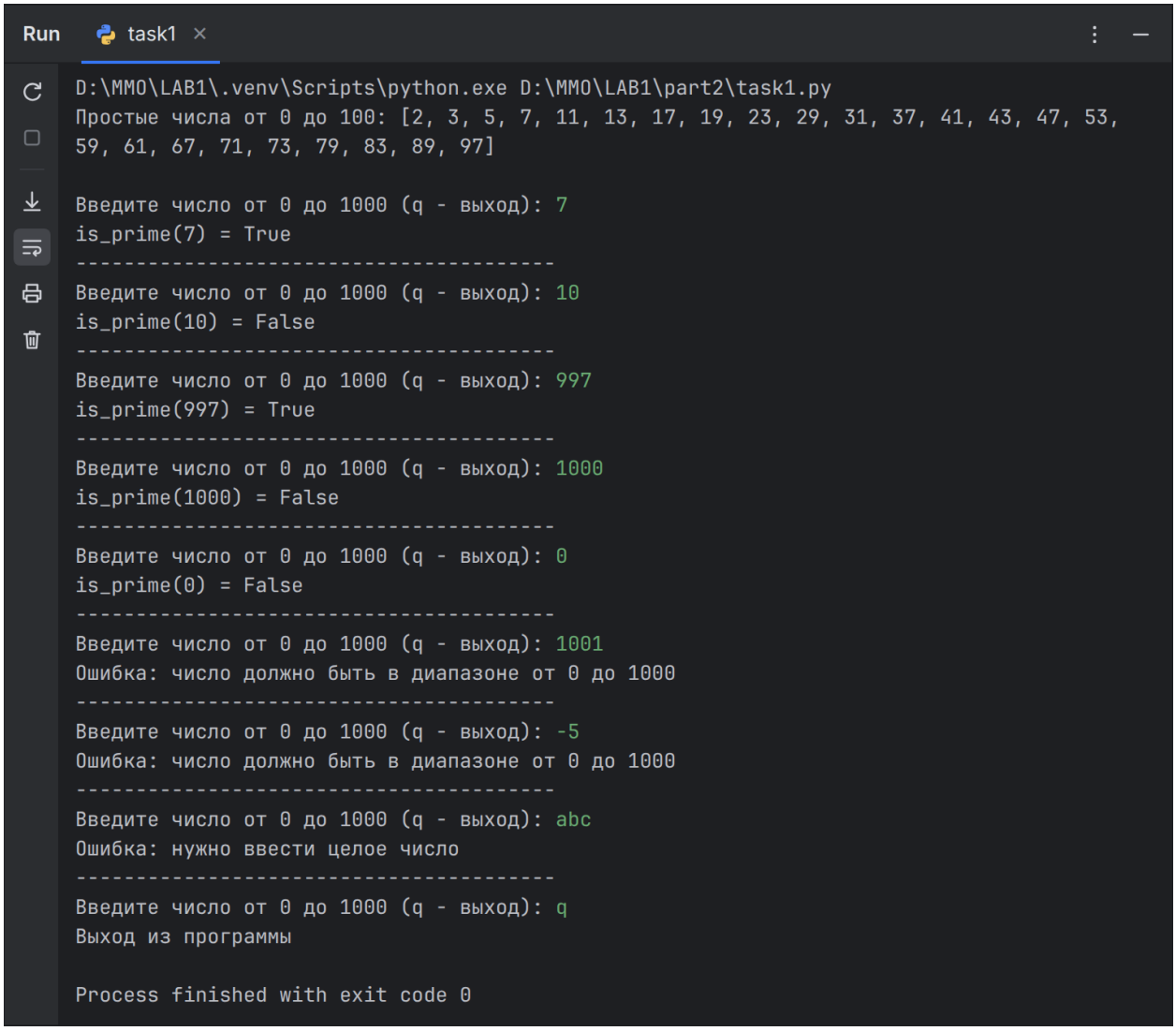


Рисунок 5.12 - Проверка чисел функцией is_prime

Проверка функции для аргументов разных типов приведена на рисунке 5.13. Для неподдерживаемых типов и словаря с несравнимыми значениями генерируется исключение TypeError, которое обрабатывается в основной программе.

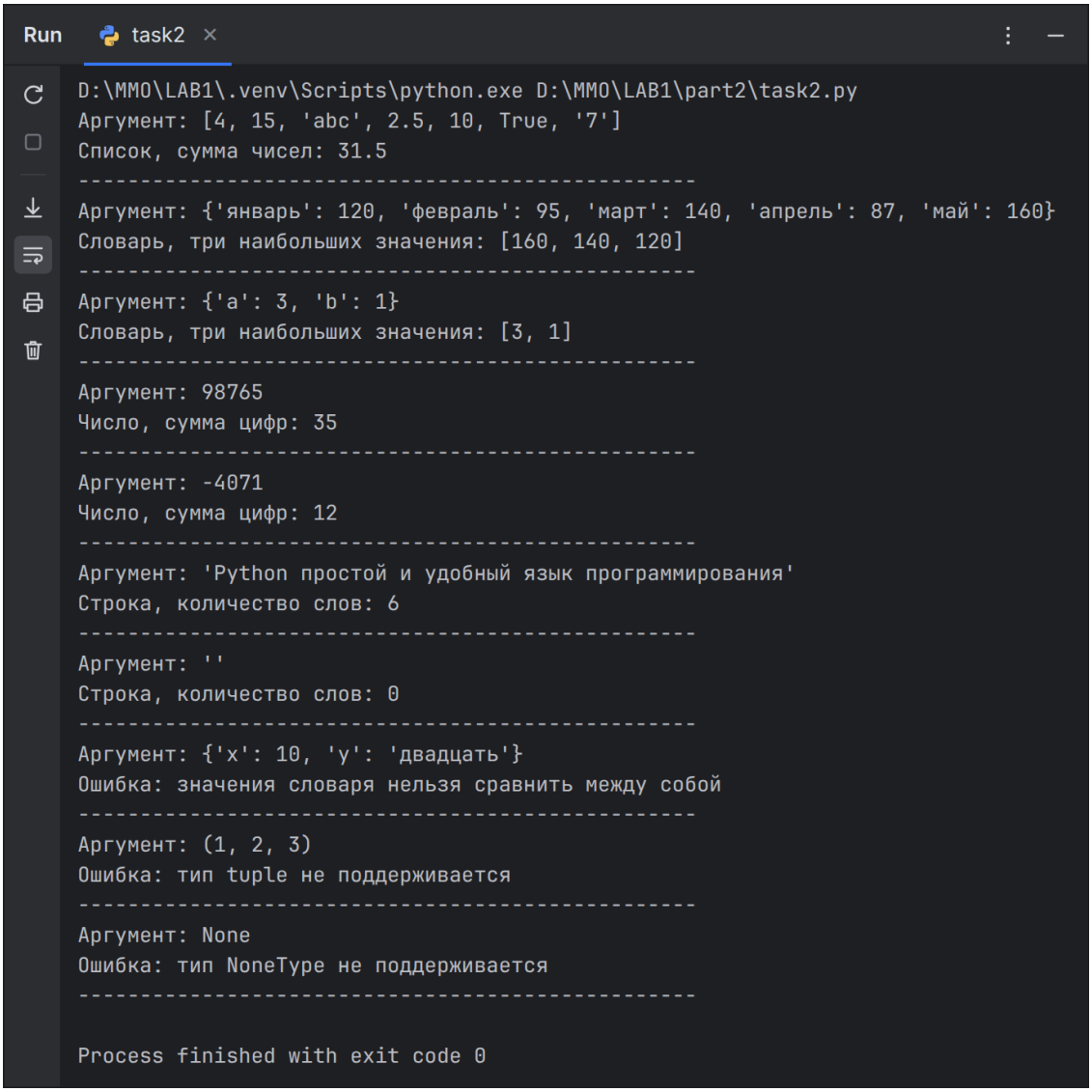


Рисунок 5.13 - Обработка аргументов разного типа

На рисунках 5.14 - 5.16 показан поиск первой строки матрицы, все элементы которой положительны, при ручном вводе матрицы, при случайном заполнении и при неверном вводе размеров.

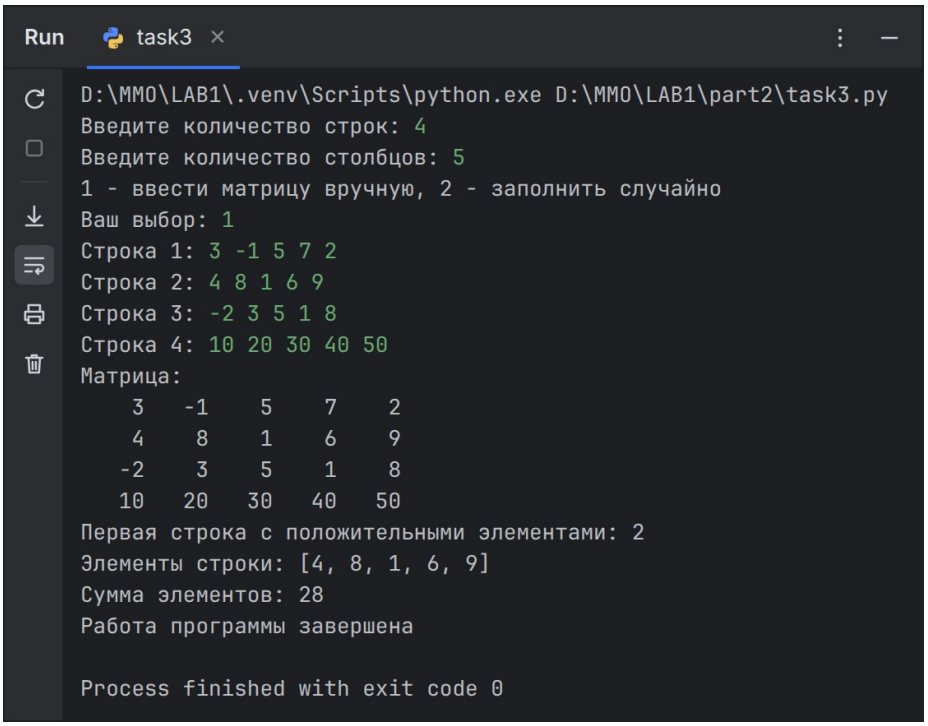


Рисунок 5.14 - Поиск строки в матрице, введенной вручную

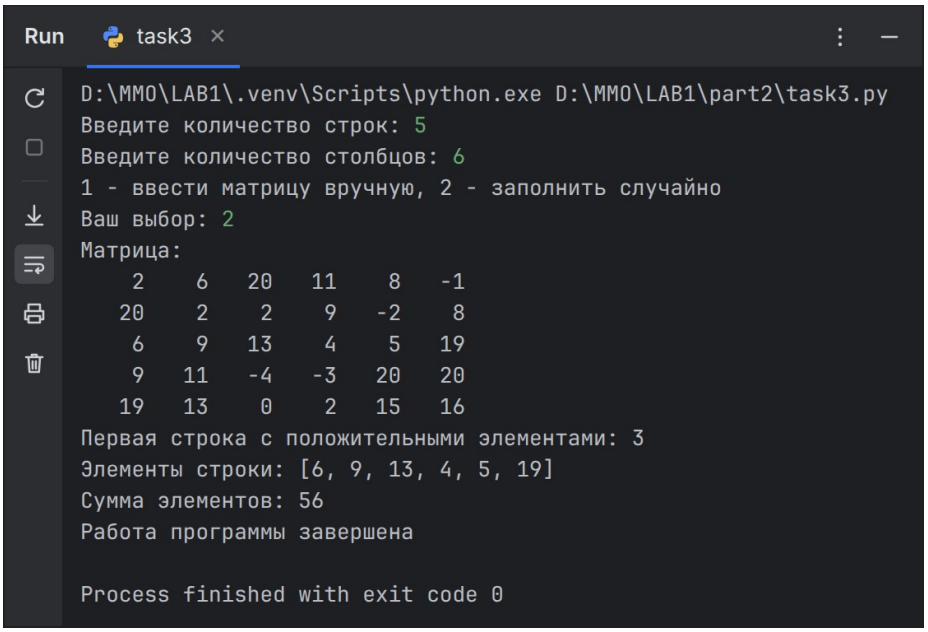


Рисунок 5.15 - Поиск строки в случайной матрице

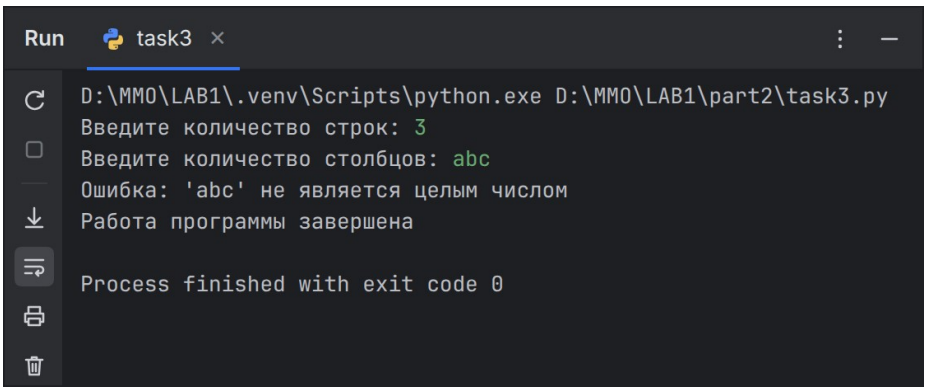


Рисунок 5.16 - Обработка неверного ввода размеров матрицы

Демонстрация работы конструкции try/except/finally приведена на рисунке 5.17. Видно, что блок finally выполняется как при успешном выполнении, так и при возникновении исключения.

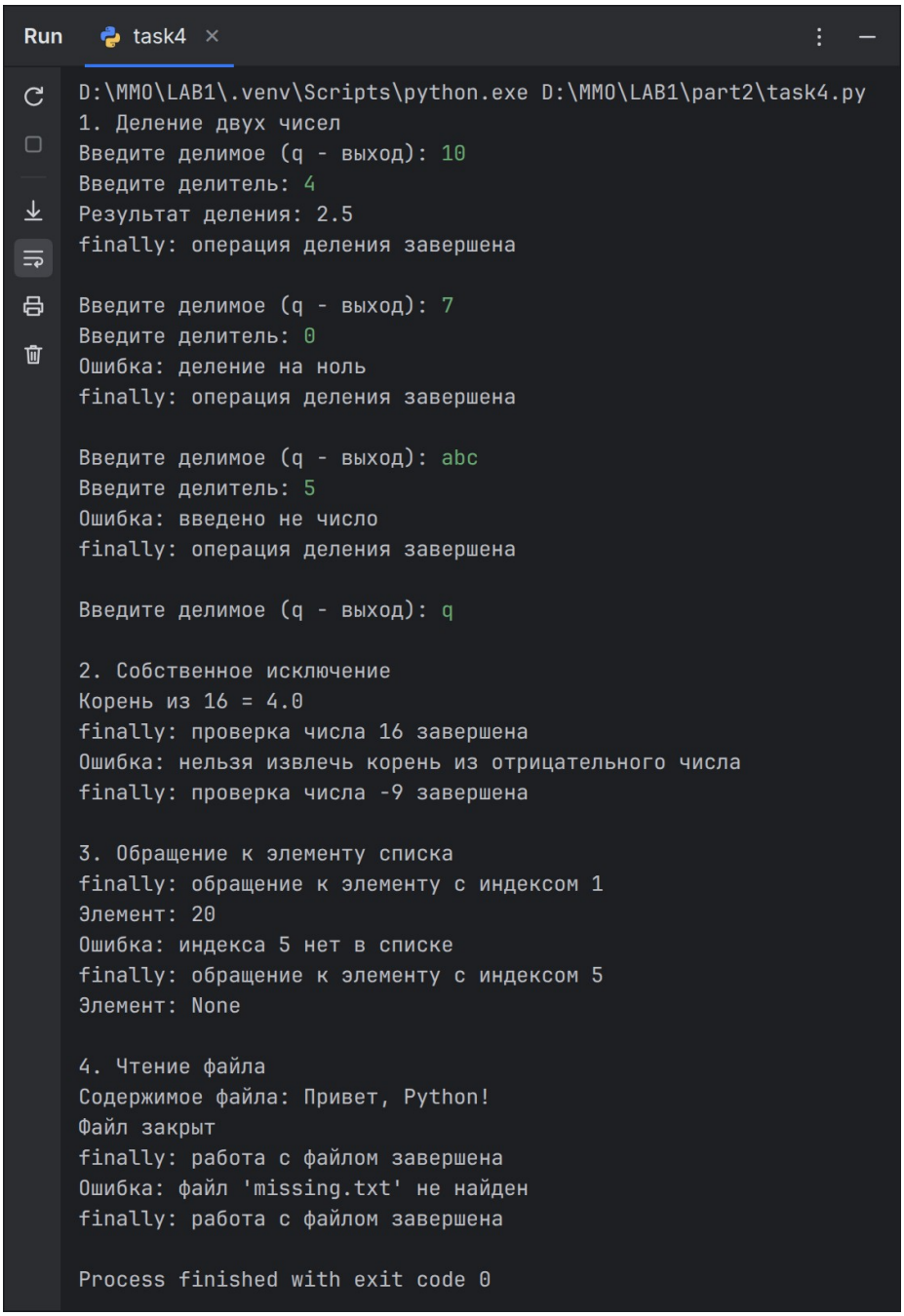


Рисунок 5.17 - Демонстрация работы try/except/finally

5.3 Часть 3

На рисунке 5.18 показана работа методов класса String для строки, введенной с клавиатуры.

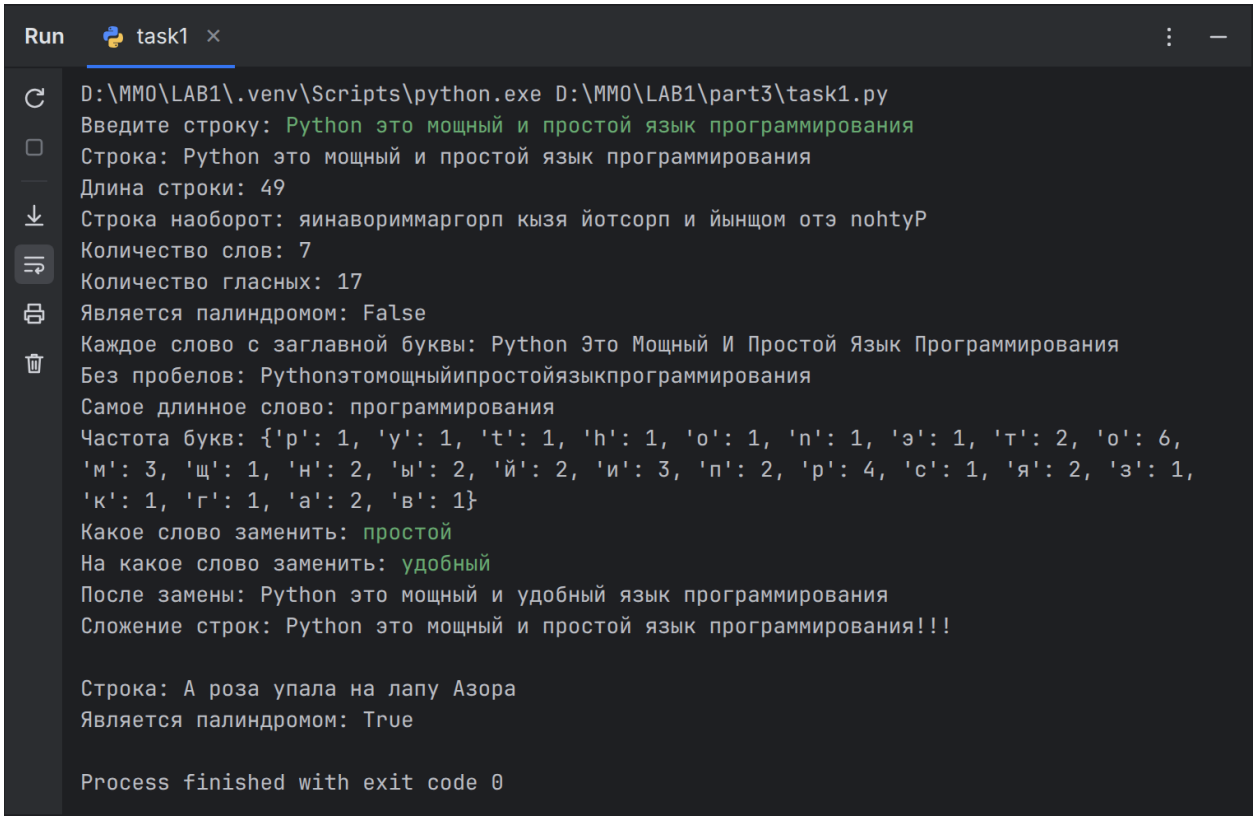


Рисунок 5.18 - Работа методов класса String

На рисунке 5.19 приведен результат создания экземпляров класса Position и вызова их методов.

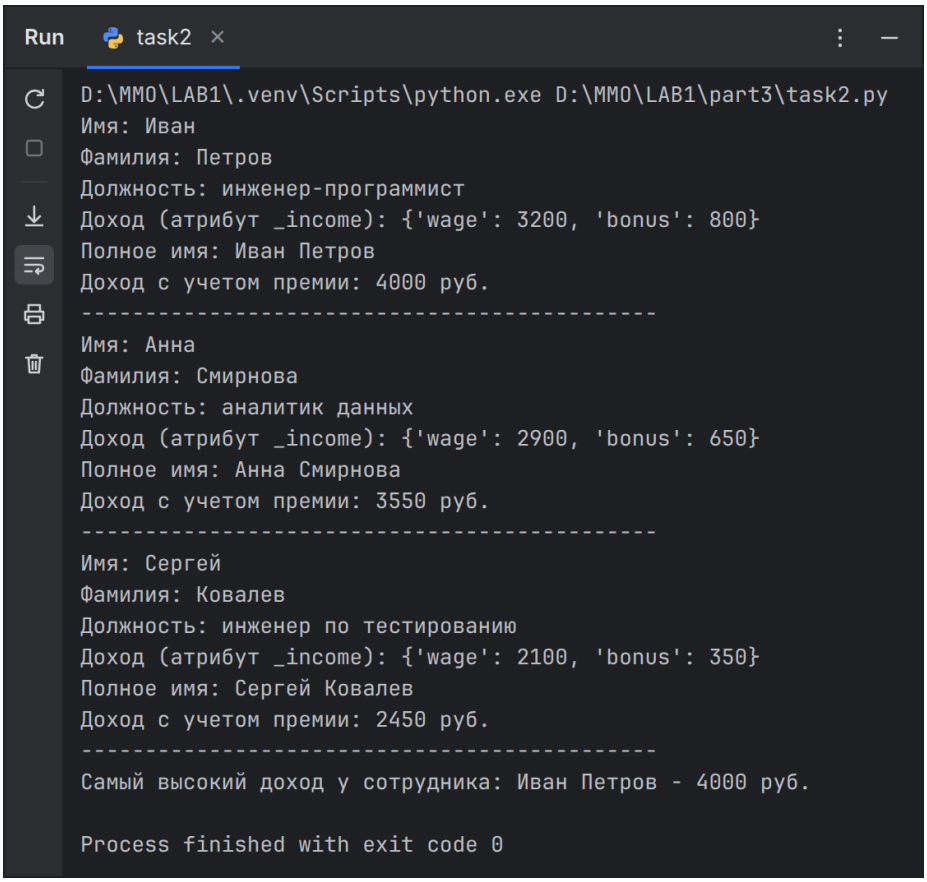


Рисунок 5.19 - Работа классов Worker и Position

Расчет времени и стоимости поездок для разных видов транспорта показан на рисунке 5.20, содержимое файла trips.txt, в который записывается информация о поездках, на рисунке 5.21.

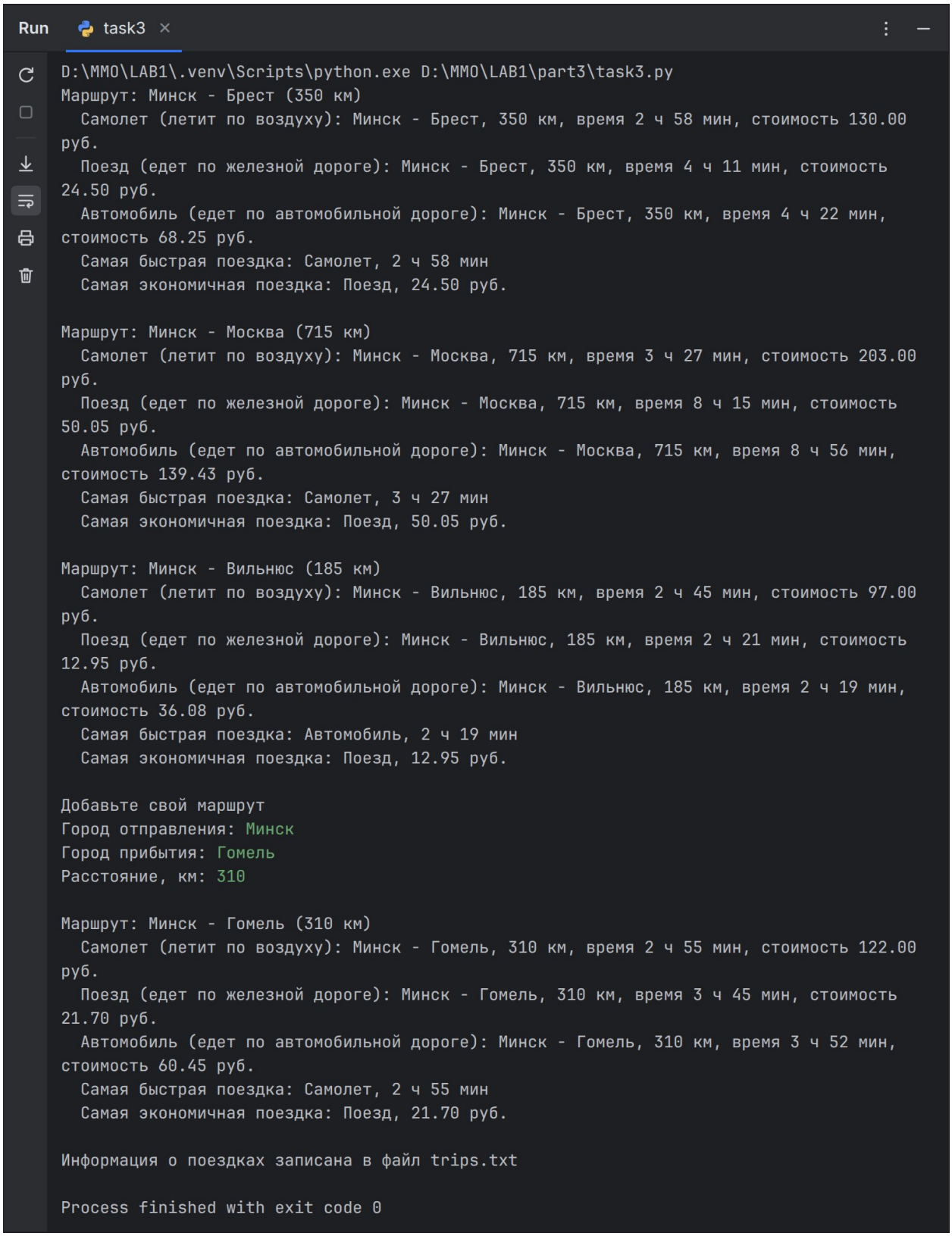


Рисунок 5.20 - Расчет времени и стоимости поездок

trips.txt

```
Самолет (летит по воздуху): Минск - Брест, 350 км, время 2 ч 58 мин, стоимость 130.00 руб.
Поезд (едет по железной дороге): Минск - Брест, 350 км, время 4 ч 11 мин, стоимость 24.50 руб.
Автомобиль (едет по автомобильной дороге): Минск - Брест, 350 км, время 4 ч 22 мин, стоимость 68.25 руб.
Самая быстрая поездка: Самолет, 2 ч 58 мин
Самая экономичная поездка: Поезд, 24.50 руб.

Самолет (летит по воздуху): Минск - Москва, 715 км, время 3 ч 27 мин, стоимость 203.00 руб.
Поезд (едет по железной дороге): Минск - Москва, 715 км, время 8 ч 15 мин, стоимость 50.05 руб.
Автомобиль (едет по автомобильной дороге): Минск - Москва, 715 км, время 8 ч 56 мин, стоимость 139.43 руб.
Самая быстрая поездка: Самолет, 3 ч 27 мин
Самая экономичная поездка: Поезд, 50.05 руб.

Самолет (летит по воздуху): Минск - Вильнюс, 185 км, время 2 ч 45 мин, стоимость 97.00 руб.
Поезд (едет по железной дороге): Минск - Вильнюс, 185 км, время 2 ч 21 мин, стоимость 12.95 руб.
Автомобиль (едет по автомобильной дороге): Минск - Вильнюс, 185 км, время 2 ч 19 мин, стоимость 36.08 руб.
Самая быстрая поездка: Автомобиль, 2 ч 19 мин
Самая экономичная поездка: Поезд, 12.95 руб.

Самолет (летит по воздуху): Минск - Гомель, 310 км, время 2 ч 55 мин, стоимость 122.00 руб.
Поезд (едет по железной дороге): Минск - Гомель, 310 км, время 3 ч 45 мин, стоимость 21.70 руб.
Автомобиль (едет по автомобильной дороге): Минск - Гомель, 310 км, время 3 ч 52 мин, стоимость 60.45 руб.
Самая быстрая поездка: Самолет, 2 ч 55 мин
Самая экономичная поездка: Поезд, 21.70 руб.
```

Рисунок 5.21 - Содержимое файла trips.txt

На рисунке 5.22 показана работа класса Student, в котором реализованы методы экземпляра, статические методы и методы класса.

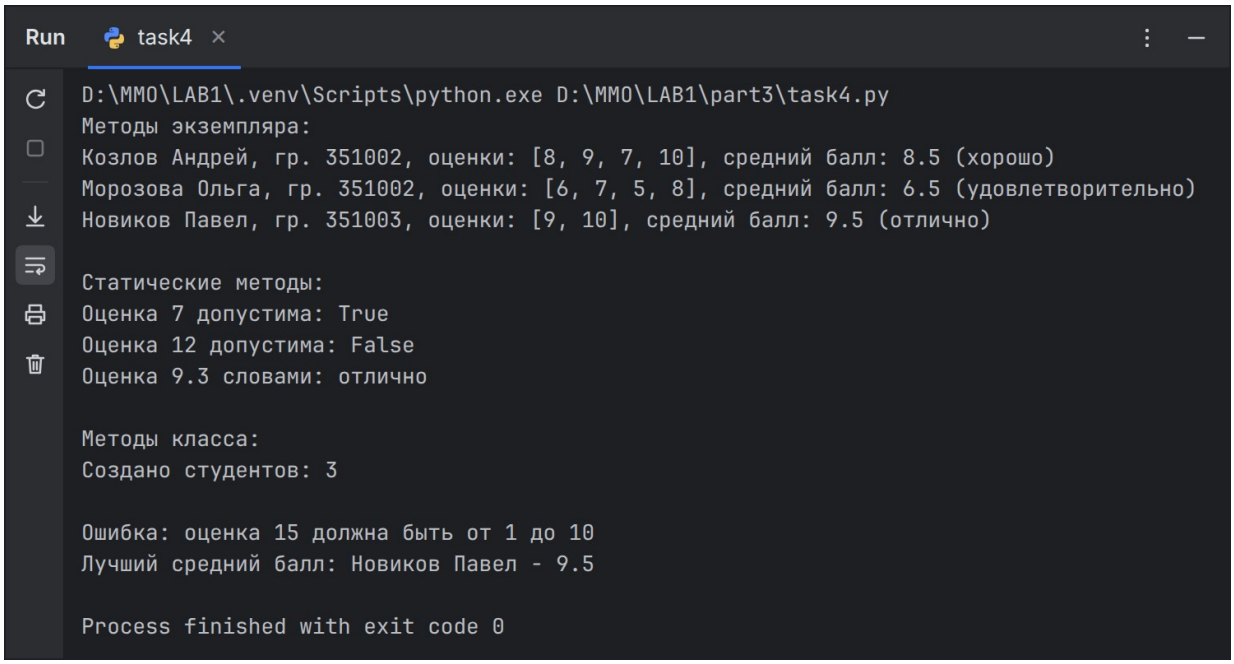


Рисунок 5.22 - Работа класса Student

6 ВЫВОДЫ

В ходе выполнения лабораторной работы были изучены основные конструкции языка Python и получены практические навыки их использования.

В первой части работы были решены задачи на работу со строками, списками, кортежами и словарями. Вычисление числа π по формуле Грегори для 500 членов ряда дало значение 3,13959, которое отличается от точного примерно на 0,002, что показывает медленную сходимость этого ряда. Также была реализована программа «Магазин автозапчастей» с меню из шести пунктов, в которой данные о товарах хранятся в словаре, а при покупке проверяется наличие товара и правильность введенного количества.

Во второй части все задачи были оформлены в виде функций с обработкой исключений. Использование блоков try/except/finally позволило программам правильно реагировать на неверный ввод пользователя (буквы вместо чисел, число вне допустимого диапазона, деление на ноль, отсутствующий файл) и не завершаться аварийно.

В третьей части были изучены основы объектно-ориентированного программирования. Были созданы класс String с методами для работы со строками, классы Worker и Position с защищенным атрибутом, иерархия классов транспортных средств с переопределением метода способа передвижения и записью результатов в файл, а также класс Student, в котором используются методы экземпляра, статические методы и методы класса.

Все программы были проверены на различных входных данных и работают правильно.